

PyGeek 2026 구두발표 대본 — 윤준하

대상 파일: docs/reference/PyGeek2026_발표_윤준하_rev7.pptx (16장 · 영문 슬라이드)

일시: 2026-08-20(목) 14:10-14:30 · Oral Presentation #1 · 홍익대 와우관 2층

슬롯 20분 = 발표 15분 + Q&A 4분 + 교체 1분 · 발표 언어: 한국어

시간 배분표

장	쪽	슬라이드	구간	배정
1	—	Title	0:00-0:40	40초
2	1	Contents	0:40-1:00	20초
3	2	Introduction — 벡터 레이어	1:00-1:55	55초
4	3	Introduction — 두 가지가 함께 바뀐다 ★	1:55-3:15	80초
5	4	Introduction — 원인을 잘못 짚으면	3:15-4:10	55초
6	5	Proposed Method — 중간 조건 ★★	4:10-5:50	100초
7	6	Proposed Method — 시스템 구조	5:50-6:45	55초
8	7	Performance Evaluation	6:45-7:35	50초
9	8	Experiment Result — 저장소 ★	7:35-8:45	70초
10	9	Experiment Result — 지표 해설 ①	8:45-9:40	55초
11	10	Experiment Result — 위치 ★	9:40-11:00	80초
12	11	Experiment Result — 지표 해설 ②	11:00-11:35	35초
13	12	Experiment Result — 등가성	11:35-12:50	75초
14	13	Conclusion ★	12:50-14:25	95초
15	14	References	14:25-14:35	10초
16	15	Thank You	14:35-14:50	15초

중간 점검: 슬라이드 8(Performance Evaluation)을 넘길 때 **7분 30초**면 정상. 8분 30초가 넘었으면 아래 <생략 가능>을 버린다.

배정에는 여유가 들어 있다. 대본을 쉬지 않고 읽으면 약 12분이고, 나머지 3분은 호흡·그림 짚기·슬라이드 넘김 뒀다. 시계가 배정보다 앞서 있어도 서두를 필요가 없다. 여유가 가장 많이 배정된 곳은 **6번(중간 조건)**과 **11번(위치 요인)** 이고, 이 두 장은 천천히 가라고 일부러 늘려 둔 것이다.

1 Title

0:00-0:40

안녕하십니까. 성결대학교 윤준하입니다.

발표에 앞서 좋은 자리를 마련해 주신 조직위원회와 좌장님께 감사드립니다.

제가 발표드릴 논문의 제목은 「벡터 데이터베이스 이전에서, 저장소 교체와 임베딩 위치 효과를 분리하는 요인 분해 방법」입니다.

제목이 조금 긴데, 쉽게 말씀드리면 이렇습니다. **데이터베이스를 옮겼더니 성능이 달라졌을 때, 무엇 때문에 달라진 것인지를 가려내는 방법**에 관한 이야기입니다.

슬라이드는 영문으로 준비했고, 발표는 한국어로 진행하겠습니다.

2 Contents

0:40-1:00

발표는 다섯 부분으로 진행하겠습니다.

먼저 **어떤 문제가 있는지** 말씀드리고, 그 문제를 가려내기 위해 저희가 **제안하는 방법**을 설명드리겠습니다.

이어서 **실험 설정과 결과**를 보여 드리고, **결론**으로 마무리하겠습니다.

3 Introduction — 벡터 레이어

1:00-1:55

먼저 배경입니다.

RAG 시스템은 크게 세 부분으로 되어 있습니다. **글을 숫자 목록으로 바꾸는 임베딩 모델, 그 숫자를 쌓아 두고 비슷한 것을 찾아 주는 저장소, 그리고 찾아온 근거를 읽고 답을 쓰는 언어 모델**입니다.

(그림 위쪽을 짚으며) 이 가운데 앞의 두 부분, 그러니까 **벡터를 다루는 부분이 서비스를 띄운 뒤에 가장 자주 갈아 끼우게 되는 자리**입니다.

(그림 아래쪽을 짚으며) 가장 흔한 경로는 이렇습니다. 지금은 일반 데이터베이스에 벡터 기능을 얹어서 쓰고 있고, 여기서 **임베딩 계산까지 데이터베이스가 직접 해 주는 제품**으로 옮겨 가는 경우입니다.

문제는 여기서 시작됩니다.

이렇게 한 번 옮길 때, **바뀌는 것이 하나가 아니라 두 개입니다.**

첫째는 **저장소**입니다. 벡터를 담아 두는 엔진이 바뀌고, 비슷한 것을 찾는 방식도 바뀝니다. 기존 것은 빠르게 찾는 대신 가끔 놓치는 방식이고, 새 것은 전부 훑어보는 방식입니다.

둘째는 **임베딩을 어디서 계산하느냐**입니다. 데이터베이스 바깥에서 하던 것을, 데이터베이스 안에서 하게 됩니다.

(그림의 붉은 두 줄을 짚으며) 화면에서 붉게 표시된 두 줄이, 옮기기 전후에 실제로 달라지는 항목입니다.

그런데 이 둘이 왜 꼭 같이 움직이는지도 말씀드려야 할 것 같습니다.

데이터를 우리 경계 안에 두려고 이전을 결정하는 경우가 많습니다. 그러면 질의가 밖으로 나가는 것도 막고 싶어지니까, 임베딩도 자연스럽게 안으로 들어옵니다.

그런데 **데이터베이스 안에 넣을 수 있는 모델은 크기 제한이 있습니다.** 그래서 안으로 옮기는 순간, 대체로 더 작은 모델을 쓰게 됩니다.

결국 옮기기 전후만 비교해서는, **달라진 원인이 저장소인지 모델인지 짚을 수가 없습니다.**

5 Introduction — 원인을 잘못 짚으면

3:15-4:10

이게 왜 실무에서 문제가 되는지 말씀드리겠습니다.

(왼쪽 상자) **품질이 떨어진 경우**, 저장소 탓인지 모델 탓인지 알 수가 없습니다.

(오른쪽 상자) **아무 변화가 없는 경우**도 사실 마찬가지입니다. 정말 아무 영향이 없었던 것인지, 아니면 한쪽이 올리고 한쪽이 내려서 서로 상쇄된 것인지 구분이 안 됩니다.

두 경우 모두 "모르겠다"는 같은 결론에 도달합니다.

그러면 처방이 어긋납니다. 저장소를 범인으로 지목하고 되돌려 놓아도, **품질은 돌아오지 않습니다.**

〈생략 가능〉 그리고 이렇게 내린 결론은 인스턴스 등급이나 모델 크기 제한이 다른 환경에는 그대로 가져다 쓸 수도 없습니다.

이 발표의 핵심입니다. 여기서 한 번 호흡을 두고, 그림을 짚으면서 천천히 갑니다.

그래서 저희가 제안드리는 것이 **브리지 조건**, 우리말로 하면 **중간 조건**입니다.

생각 자체는 단순합니다. **A에서 C로 한 번에 건너뛰지 말고, 중간에 B라는 정거장을 하나 만드는 것입니다.** 만드는 방법도 간단합니다. **지금 쓰고 있는 시스템이 이미 만들어 둔 벡터를, 그대로 새 저장소에 실어 올립니다.** 화면 가운데 있는 조건 B가 그것입니다.

그러면 조건 B는 **저장소만 새것이고, 임베딩 모델과 임베딩 위치는 원래 그대로인 상태**가 됩니다.

그 결과 (왼쪽 화살표를 가리키며) **A에서 B로 갈 때는 저장소만 달라지고**, (오른쪽 화살표를 가리키며) **B에서 C로 갈 때는 임베딩 위치만 달라집니다.** 한 번에 하나씩만 보게 되는 것입니다.

실무적으로 좋은 점을 하나 말씀드리고 싶습니다. **벡터를 다시 만들지 않아도 됩니다.** 이미 있는 것을 옮겨 싣기만 하면 되니까, 임베딩 비용이 더 들지 않고 **운영 중인 서비스를 멈추지 않아도 됩니다.**

이전을 결정하기 전에 미리 해 볼 수 있다는 점이, 저희가 생각하는 이 방법의 쓸모입니다.

실제로 무엇이 달라지는지 구조로 보여 드리겠습니다.

(왼쪽) 지금 시스템은 애플리케이션이 **바깥에 있는 임베딩 서비스를 불러서** 결과를 받아 오고, 그것을 다시 저장소로 보냅니다. **밖으로 두 번 나갔다 옵니다.**

(오른쪽) 옮기고 나면 임베딩과 검색이 모두 데이터베이스 안에서 끝납니다. **밖으로 나가는 것은 한 번으로 줄고, 외부 모델 호출은 아예 없어집니다.**

한 가지 꼭 말씀드릴 것이 있습니다. **검색 규칙과 설정값은 세 조건에서 완전히 똑같이 고정했습니다.** 비교하려는 것 말고 다른 게 섞이면 안 되기 때문입니다.

여기서 7분 30초면 정상 속도입니다.

실험 설정입니다.

대상은 운영 중인 국내 제품 리뷰 분석 서비스의 리뷰 조각 1,347개이고, 미리 정해 둔 질문 43개로 측정했습니다. 질문은 세 조건에 똑같이 던졌습니다.

지표는 세 가지입니다. 검색 결과가 얼마나 같은지, 찾아온 근거가 질문에 얼마나 맞는지를 0점에서 2점으로 매긴 값, 그리고 검색에 걸린 시간입니다.

(표를 짚으며) 표에서 파란 칸이 저장소 요인으로 바뀐 항목, 붉은 칸이 위치 요인으로 바뀐 항목입니다. 색이 없는 칸은 세 조건에서 그대로입니다.

<생략 가능> 신뢰구간은 가진 데이터를 만 번 다시 뽑아 보면서 값이 얼마나 흔들리는지를 재서 구했고, 측정은 가장 낮은 등급 인스턴스에서 했습니다.

9 Experiment Result — 저장소 요인 ★

7:35-8:45

첫 번째 결과입니다. 저장소만 바꾼 A에서 B 구간입니다.

결론부터 말씀드리면, 품질 변화가 검출되지 않았습니다.

상위 10개 결과가 얼마나 겹치는지를 보면 0.971이 나왔습니다. 거의 같은 목록이 돌아왔다는 뜻입니다.

풀어서 말씀드리면, 같은 벡터를 실었을 때 두 저장소는 질문 43개 가운데 37개에서 상위 10개가 정확히 똑같았습니다.

근거가 질문에 얼마나 맞는지도 비교했는데, p값이 0.523으로 나왔습니다. 차이라고 부를 만한 것이 나오지 않았다는 뜻입니다. p값은 다음 장에서 설명드리겠습니다.

나머지 6개에서 차이가 난 이유는, 기존 시스템이 빠르게 찾는 대신 가끔 놓치는 방식을 쓰기 때문입니다. 놓친 양이 대략 1.6퍼센트 정도였습니다.

<생략 가능> 점수가 같은 질문을 빼고 다시 검정해 보아도 결론은 같았습니다.

여기가 숫자를 처음 자세히 다루는 자리입니다. 서두르지 말고 천천히 갑니다.

방금 나온 숫자 두 개를 잠깐 설명드리겠습니다.

(왼쪽) 첫 번째는 **두 결과 목록이 얼마나 겹치는지**입니다. 완전히 같으면 1, 하나도 안 겹치면 0입니다. 지금은 A와 B가 거의 같은 목록을 가져왔기 때문에 1에 가깝게 나왔습니다.

(오른쪽) 두 번째가 **p값**입니다. 이렇게 보시면 됩니다.

먼저 **"사실은 아무 차이도 없다"**고 가정합니다. 그리고 두 조건의 점수를 뒤섞어서 **가짜 차이를 수천 번 만들어 봅니다**. 그렇게 만든 가짜 차이들 가운데 **지금 관측한 것만큼 큰 것이 얼마나 흔한지**, 그 비율이 p값입니다.

흔하면 우연으로 설명이 되고, 아주 드물면 우연이라고 보기 어렵습니다.

(오른쪽 그림을 짚으며) 지금은 관측값이 **가짜 차이들의 한가운데**에 있습니다. 우연으로 충분히 설명이 되는 자리입니다.

같은 잣대를 다음 장에서 위치 요인에 그대로 대 보겠습니다.

11 Experiment Result — 위치 요인 ★

9:40-11:00

두 번째 결과입니다. **임베딩 위치만 바꾼 B에서 C 구간**입니다.

여기서는 결과가 확연히 달라집니다.

방금 0.971이었던 겹침이 **0.175로 떨어졌습니다**. 거의 딴 목록이 돌아왔다는 뜻입니다.

p값도 **0.003**입니다. 우연으로 보기는 어려운 수준입니다. 차이의 크기 자체도 중간 정도로 나왔습니다.

강조드리고 싶은 것은 이것입니다. **저장소도 같고 검색 방식도 같은데, 같은 질문에 대체로 다른 근거가 돌아왔습니다**.

그리고 앞 구간에서 저장소 효과가 검출되지 않았기 때문에, **관측된 품질 변화는 사실상 전부 임베딩 위치, 그리고 거기에 달려 온 모델 크기 제한에서 나온 것으로 볼 수 있습니다**.

여기서 한 박자 쉬고, 청중을 보며 말합니다.

만약 앞뒤만 비교했다면 어땠을까요. **저장소 탓으로 돌렸을 것입니다**. 그리고 저장소를 되돌렸겠지만, 품질은 돌아오지 않았을 것입니다.

앞 장의 두 지표를 위치 요인에 그대로 적용한 그림입니다.

(왼쪽) 겹침이 0.971에서 0.175로 떨어졌습니다. 두 목록을 합치면 17개인데, 그중 겹치는 것은 3개뿐입니다.

(오른쪽) p값은 0.523에서 0.003으로 내려갔습니다. 관측값이 **가짜 차이들 거의 전부보다 바깥에** 나가 있습니다.

같은 잣대를 두 구간에 똑같이 댔는데 이만큼 다른 그림이 나온다는 것이 이 방법의 요점입니다.

13 Experiment Result — 등가성 검정

11:35-12:50

여기서 한 가지 조심스럽게 말씀드릴 부분이 있습니다.

앞에서 저장소 요인은 차이가 검출되지 않았다고 말씀드렸습니다. 그런데 **"차이를 못 찾았다"**와 **"차이가 없다"**는 **다른 말입니다**. 못 찾은 것은 데이터가 부족해서일 수도 있습니다.

그래서 **"차이가 없다"**를 주장하려면 **그것대로 검정을 따로 해야 합니다**. 저희는 등가성 검정을 수행했습니다. 2점 만점에서 플러스마이너스 0.15점 안쪽이면 사실상 같다고 보자고 기준을 정했습니다.

(그림의 파란 띠를 짚으며) 화면의 열은 띠가 그 기준입니다. 신뢰구간이 이 띠 안에 통째로 들어와야 **"같다"**고 말할 수 있는데, 보시는 것처럼 왼쪽 끝이 조금 나가 있습니다.

결과는 p값 0.052였습니다. 기준선인 0.05를 **아슬아슬하게 넘지 못했습니다**.

그래서 저희는 이렇게 보고합니다. **"같다는 것이 입증되었다"**가 아니라, **"차이를 찾지 못했고, 있더라도 이 정도 크기 안쪽이다"**입니다.

다만 구간이 뻗은 방향은 **새 저장소 쪽 점수가 더 높은 방향**입니다. 따라서 **저장소를 바꿔서 품질이 나빠졌**다고 볼 근거는 없습니다.

마지막 장입니다. 정리 부분과 판단 기준 사이에서 한 번 호흡을 돕니다.

정리하겠습니다.

방법 면에서는, 중간에 조건 하나를 더 두는 것만으로 앞뒤 비교 한 번이 한 요인씩 보는 비교 두 번으로 나뉩니다.

이 서비스에서 관측한 것은, 저장소를 바꾼 것만으로는 품질 변화가 검출되지 않았고, 관측된 변화가 전부 임베딩 위치에서 나왔다는 점입니다.

저희가 이 발표에서 드리고 싶은 문장은 이것입니다. 임베딩을 안으로 들이는 것은 성능을 사는 거래가 아니라, 데이터를 경계 안에 두는 대가로 성능을 내주는 거래입니다.

실무 판단 기준으로 옮기면 이렇게 됩니다.

저장해 둔 데이터를 우리 경계 안에 두는 것만이 목적이라면, 저장소만 바꾸는 것으로 충분합니다.

질의가 나가는 경로까지 안에 두어야 한다면, 옮기기 전에 모델 크기가 충분한지를 먼저 확인하셔야 합니다. 중간 조건은 그 확인을 옮기기 전에 할 수 있게 해 줍니다.

마지막으로 한계를 분명히 말씀드리겠습니다. 한 제품 도메인에서 나온 한국어 데이터 하나로 얻은 결과이고, 근거 채점에 사람 평가자가 참여하지 않았습니다. 측정도 가장 낮은 등급 인스턴스에서 했기 때문에 시간 수치는 다른 장비로 일반화할 수 없습니다.

또 설계상 위치와 모델 크기가 붙어 있습니다. 이 둘을 떼어 놓으려면, 작은 모델을 데이터베이스 바깥에서 돌리는 네 번째 조건이 필요합니다. 다음 과제로 생각하고 있습니다.

15 References

14:25-14:35

읽지 않습니다. 화면만 띄우고 한 문장으로 넘깁니다.

본 연구에서 참고한 문헌입니다.

16 Thank You

14:35-14:50

이상으로 발표를 마치겠습니다.

끝까지 들어 주셔서 감사합니다. 질문 주시면 아는 범위에서 성실히 답변드리겠습니다.

<시간이 남을 때만> 화면에 제 메일 주소를 띄워 두었습니다. 발표 후에 물어보실 것이 있으시면 편하게 연락 주십시오.

예상 질문과 답변

답변은 짧게 시작해서 필요하면 덧붙이는 순서로 갑니다. 모르는 것은 모른다고 말하는 편이 낫습니다.

Q1. 왜 더 큰 임베딩 모델을 쓰지 않았습니까?

데이터베이스 안에서 모델을 돌리려면, 그 데이터베이스가 감당할 수 있는 크기 안에서 골라야 합니다. 저희가 쓴 384차원 모델은 그 제한 안에서 고를 수 있었던 선택지였습니다.

그리고 이 제약 자체가 저희 논문이 다루는 대상이기도 합니다. 위치를 안으로 옮기면 모델 크기가 따라온다는 점이, 바로 실무에서 문제가 되는 지점이라고 보았습니다.

Q2. 질문 43개는 너무 적지 않습니까?

말씀하신 대로 표본이 크지 않습니다. 한계로 인정합니다.

그래서 저희는 값 하나만 제시하지 않고 **신뢰구간과 차이의 크기를 함께 보고했고**, 결론도 그 구간 안에서만 이야기하려고 했습니다. 질문 수를 늘리는 것은 저희도 필요하다고 보고 있습니다.

Q3. 관련성 점수를 사람이 매기지 않은 것은 신뢰할 수 있습니까?

한계로 명시했습니다. 사람 평가자가 참여하지 않았습니다.

다만 채점 기준을 세 조건에 똑같이 적용했기 때문에, **조건끼리 비교한다는 목적에서는 방향성이 유지된다고 보았습니다**. 점수의 절대값 자체를 주장하려는 것은 아닙니다. 사람 평가를 덧붙이는 것은 다음 과제로 생각하고 있습니다.

Q4. 중간 조건이라는 것이 새로운 방법입니까?

구성 요소 하나하나가 새롭다고 말씀드리기는 어렵습니다. 벡터를 옮겨 싣는 것도, 요인을 하나씩 바꿔 보는 것도 익숙한 방식입니다.

저희가 기여라고 생각하는 부분은, **벡터 데이터베이스 이전이라는 구체적인 의사결정 상황에서 이것을 하나의 절차로 정리하고, 벡터를 다시 만들지 않고 운영 서비스를 멈추지 않고도 할 수 있다는 것을 실제 측정으로 보인 점입니다**.

Q5. 그러면 이전을 하지 말라는 뜻입니까?

그런 뜻은 아닙니다.

저장해 둔 데이터를 경계 안에 두는 것이 목적이라면, 저장소만 바꾸는 선택으로 충분하다는 것이 저희 결과입니다. 이 경우에는 품질 변화가 검출되지 않았습니다.

질의 경로까지 안에 두고 싶으시다면, **그때는 모델 크기가 충분한지를 먼저 확인하시라**는 것이 저희가 드리는 말씀입니다.

Q6. 검색 방식 차이가 결과에 영향을 주지는 않았습니까?

영향이 있었고, 그 크기를 재 보았습니다. 기존 시스템은 빠르게 찾는 대신 가끔 놓치는 방식이고 새 시스템은 전부 훑는 방식인데, 그 차이로 기존 시스템이 놓친 부분이 **약 1.6퍼센트** 수준이었습니다.

이 차이를 포함해도 저장소 요인에서는 유의한 품질 변화가 검출되지 않았습니다.

Q7. p값 0.052면 사실상 같다고 봐도 되는 것 아닙니까?

기준선을 넘지 못한 것은 사실이라, 저희는 "같다"고 주장하지 않기로 했습니다.

다만 말씀하신 대로 아주 근소한 차이이고, 신뢰구간이 뺀 방향도 새 저장소 쪽이 유리한 방향입니다. 그래서 **"품질이 나빠졌다고 볼 근거는 없다"**까지는 말씀드릴 수 있다고 보았습니다. 질문 수를 늘리면 판정이 달라질 여지가 있다고 생각합니다.

발표 전 확인 사항

- ☐ **점심시간(12:00-13:30)에** 강연장에서 파일 이관 + 발표자 도구 가능 여부 확인 (13:30 이후에는 세션이 연속이라 PC를 만질 틈이 없음)
- ☐ USB에 **rev7 발표용 + PDF 백업**
- ☐ 이 대본 A4 출력 (발표자 도구를 못 쓸 경우의 1차 수단)
- ☐ 14:05까지 앞자리 이동
- ☐ 좌장이 교신저자 성함으로 소개할 수 있음 — 첫 문장에서 본인 이름을 분명히 밝힐 것